2026/9/8 12:14:31

setState 是如何 感知 他所在 上下文环境的 更新优先级的?

2026/9/8 12:14:31

User • 12:07

现在有一个问题, 好像 click点击, change监听中的setCount 更新和 startTransition中的setCount 更新具有不同的优先级.

所以问题就是 setState函数如何知道 他所在的环境?
特别是 存在异步await的情况, 比如click中 先await fetch 之后 再setState，相比click中直接 setState的情况，前者 setState的执行环境是执行完fetch之后系统的自然调度(click早已经在await fetch 的时候就执行结束了), 而后者的setState的执行是处于click调度中的.

Model • 12:07

Thoughts

Expand to view model thoughts

这是一个极为深刻的底层问题！你已经触及到了 React 调度系统中**最核心的“环境感知”与“上下文追踪”机制**。

你的疑惑非常直击本质：`setState` 只是一个普通的函数调用，你也没有给它传参告诉它“我是从哪里来的”，那么：**它到底是怎么知道自己处于什么优先级的？在跨越了 `await` 这个异步鸿沟之后，它又是怎么感知到环境变化的？**

答案的谜底，藏在 React 源码中的几个**“全局共享变量（模块级变量）”**里。

React 底层通过一个叫做 `requestUpdateLane(fiber)` 的核心函数来为 `setState` 分配优先级。这个函数会去读取当前的全局环境变量。

我们把你的问题拆解为三个最典型的场景，来看看 React 是如何通过操纵全局变量，完成这场“移花接木”的。

场景一：直接在 `click` 中同步 `setState`

（它是如何知道自己是最高优先级的？）

React 有自己的一套**合成事件系统（Synthetic Event）**。当你在按钮上绑定 `onClick` 时，浏览器原生的点击事件会被 React 的顶层监听器拦截，然后进入 React 的事件分发器。

执行流程（源码级重现）：

1. **注入高优先级标记：** React 发现这是一个点击事件（离散交互），它会把源码中的一个全局变量 `currentUpdatePriority` 设置为 `DiscreteEventPriority` **（离散优先级，对应 `SyncLane`）**。
2. **执行你的代码：** React 调用你写的 `click` 函数。
3. `setState` **获取优先级：** 执行到 `setState` 时，它内部会调用 `requestUpdateLane()` 去检查全局变量。它抬头一看，发现当前全局变量是 `DiscreteEventPriority`，于是高兴地给这次更新打上了 `SyncLane` **（最高优先级）** 的标签。
4. **清理现场：** 你的 `click` 函数执行完毕。React 的事件分发器走到了 `finally` 块，**把全局变量重置回默认值** `DefaultEventPriority`。

场景二：跨越 `await` 之后的 `setState`

（它是如何知道自己“失去”了高优先级的？）

这就来到了你问题中最核心的场景。如果在 `click` 中加入了 `await fetch`，环境感知就发生了戏剧性的断裂。

JavaScript

```javascript
async function click() {
  // 此时全局 currentUpdatePriority = DiscreteEventPriority
  setCount(1); // 获得 SyncLane

  await fetch('/api'); // 🔴 发生异步挂起，主线程交出！

  setCount(2); // 获得 DefaultLane（降级了！）
}
```

为什么会降级？因为“清理现场”提前发生了！

1. 前半部分和上面一样，React 把全局优先级设为 `DiscreteEventPriority`。
2. 当执行到 `await fetch` 时，JavaScript 引擎会把你当前的 `click` 函数挂起（Yield），并**把主线程的控制权还给 React 的外层事件分发器**。
3. React 的外层事件分发器一看：“哦，里面执行完了（其实只是挂起了）”，于是它毫不犹豫地执行了 `finally` 块，**把全局变量 `currentUpdatePriority` 重置回了 `DefaultEventPriority`**。
4. 过了 2 秒钟，网络请求回来了，浏览器微任务队列恢复执行 `await` 后面的代码。
5. 此时执行 `setCount(2)`，它再次去读取全局变量，**发现此时的变量早就被 React 还原成了 `DefaultEventPriority`**。
6. 结果：`setCount(2)` 只能无奈地给自己打上 `DefaultLane` **（默认优先级）** 的标签，失去了同步更新的特权。

总结：跨越异步鸿沟后，`setState` 之所以表现不同，不是因为它知道“我在异步里”，而是因为它原本依赖的“高优先级全局体验卡”到期被收回了。

场景三：在 `startTransition` 中的 `setState`

（它是如何知道自己是低优先级的？）

`startTransition` 的实现原理，同样是操纵全局变量，但它操纵的是另一个叫做 `ReactCurrentBatchConfig.transition` 的全局对象。

JavaScript

```javascript
function click() {
  startTransition(() => {
    setCount(3);
  });
}
```

底层执行流：
当你调用 `startTransition(callback)` 时，React 在源码里其实只做了极其简单的几件事（伪代码）：

JavaScript

```javascript
function startTransition(callback) {
  // 1. 记住之前的状态
  const prevTransition = ReactCurrentBatchConfig.transition;
  // 2. 开启低优先级上下文
  ReactCurrentBatchConfig.transition = {}; // 只要不是 null，就代表处于 transition 中

  try {
    // 3. 执行你的回调
    callback();
  } finally {
    // 4. 恢复之前的状态
    ReactCurrentBatchConfig.transition = prevTransition;
  }
}
```

当你在 `callback` 里面调用 `setState` 时，`requestUpdateLane()` 函数会**优先检查** `ReactCurrentBatchConfig.transition`。只要发现它有值，`setState` 就会直接忽略其他的事件优先级，毫不犹豫地给自己打上 `TransitionLane` **（低优先级车道）** 的标签。这样，这次更新就可以被后续的时间切片打断。

🏆 终极进化：React 19 的 Async Transition (异步过渡)

既然我们说到 `await` 会导致全局变量被重置（丢失上下文），那你在 React 19 中可能会遇到一个巨大的逻辑挑战：

在 React 19 中，`startTransition` 是支持异步的！

JavaScript

```javascript
startTransition(async () => {
  setCount(1);
  await fetch('/api');
  setCount(2); // 🔴 在 React 19 中，这依然是 TransitionLane！并没有丢失！
});
```

为什么这里跨越了 `await`，`setCount(2)` 却没有丢失 Transition 上下文呢？

这是因为 React 19 团队为了解决“异步环境丢失全局变量”的痛点，引入了更为深度的底层追踪机制（类似于 Node.js 中的 `AsyncLocalStorage` 思想）。

- 在 React 19 的源码中，如果 `startTransition` 返回了一个 Promise，React 会**接管这个 Promise 的生命周期**。
- 在每次微任务恢复执行前**，React 会再次强行把 `ReactCurrentBatchConfig.transition` 塞回去，等这段代码执行完，再把它拿走。**
- 这样就强行缝合了因为 `await` 导致的上下文断裂，保证了整个异步流中的 `setState` 统统属于低优先级过渡任务。

总结（核心脉络）

`setState` 本身是一个“瞎子”，它不属于任何组件，它也不知道自己在哪。
它唯一能做的，就是在被调用的那一瞬间，去读取 React 维护的几个**全局标记（Global Flags）**。

1. **普通的 `click`** 靠的是全局变量 `currentUpdatePriority = DiscreteEventPriority`。一旦遭遇 `await`，外层函数提前结束，全局变量被清空，后续的 `setState` 就会降级为默认优先级。
2. `startTransition` 靠的是全局对象 `ReactCurrentBatchConfig.transition`。
3. **React 的调度魔法**，本质上就是在你执行代码之前，悄悄挂上全局标记；在你代码执行完（或挂起）后，悄悄摘下标记的游戏。

Google AI models may make mistakes, so double-check outputs.

2026/9/8 12:17:31

setState 是如何 感知 他所在 上下文环境的 更新优先级的?

2026/9/8 12:17:31

核心答案：`setState` 本身并不“记住”它是从哪个事件里出来的。它只看调用那一刻的全局状态。

1. setState 如何判断优先级?

调用 `setState`（或 `useState` 的 setter）时，最终会走到 `requestUpdateLane(fiber)`。这个函数主要看两样东西:

1. **当前全局的** `executionContext`（是否在 Render/Commit 中等）
2. **当前的事件优先级**（由 React 事件系统或 `ReactDOMSharedInternals` 维护的“当前更新优先级”）

关键逻辑大致是:

```JavaScript
// 简化示意
function requestUpdateLane(fiber) {
  // 1. Legacy 模式直接 SyncLane
  if ((fiber.mode & ConcurrentMode) === NoMode) {
    return SyncLane;
  }

  // 2. 正在 render 阶段 → 特殊处理
  if ((executionContext & RenderContext) !== NoContext) {
    // ...
  }

  // 3. 最重要：看当前“事件优先级”
  return eventPriorityToLane(resolveUpdatePriority());
}
```

`resolveUpdatePriority()` 会读取 React 内部维护的**当前更新优先级**（DiscreteEventPriority / ContinuousEventPriority / DefaultEventPriority 等）。

这个优先级不是靠闭包或调用栈“记忆”的，**而是 React 在进入/退出事件处理时主动设置的全局状态**。

2. 点击事件里直接 setState 为什么是高优先级?

React 的事件系统（`dispatchDiscreteEvent` 等）在调用你的 `onClick` 之前会做类似这样的事:

```JavaScript
// 伪代码
const prevPriority = getCurrentUpdatePriority();
setCurrentUpdatePriority(DiscreteEventPriority);  // 标记：现在是离散事件

try {
  // 调用你的 click 处理函数
  yourClickHandler(event);
} finally {
  setCurrentUpdatePriority(prevPriority);  // 恢复
}
```

所以在 `onClick` **同步执行期间**调用的 `setState`，`requestUpdateLane` 会看到当前是 `DiscreteEventPriority`，于是分配 **SyncLane / 高优先级**。

3. await 之后再 setState 为什么优先级会变?

```JavaScript
async function handleClick() {
  setCount(1);          // ✅ 还在 DiscreteEventPriority 中 → 高优先级

  await fetch('/api');  // 🔴 关键点：函数挂起，事件包装器已经结束

  setCount(2);          // ❌ 此时当前更新优先级已经被恢复成默认值
}
```

发生了什么:

1. `await` 让当前函数挂起，控制权交回浏览器。
2. React 的事件包装器（`try/finally`）已经执行完毕，**把当前更新优先级恢复成了默认值**（通常是 `DefaultEventPriority`）。
3. 等 `fetch` 完成后，Promise 回调把 `setCount(2)` 推到微任务/任务队列里执行。
4. 此时调用 `setCount(2)`，`requestUpdateLane` **看到的已经是默认优先级**，不再是 Discrete。

所以:

- 直接在 click 里 `setState` → Discrete → 高优先级（SyncLane）
- `await` 之后再 `setState` → 默认优先级（更接近普通更新）

`setState` 不会“记得”自己原本是从 click 里出来的，它只看调用瞬间的全局优先级状态。

4. startTransition 是怎么做的?

`startTransition` 是显式改优先级的:

```JavaScript
startTransition(() => {
  setCount(2);  // 这里面的 setState 会被标记为 TransitionLane
});
```

它内部会临时把当前更新优先级设为 Transition，执行回调，然后再恢复。即使你在 transition 回调里再 `await`，React 19 对 Actions 做了改进，尽量保持 transition 上下文，但普通的 `startTransition(() => { await ...; setState })` 在 `await` 后仍可能丢失。

5. 总结对比

场景	调用 setState 时看到的优先级	结果
`onClick` 里直接 `setState`	DiscreteEventPriority	高优先级（SyncLane）
`onClick` 里 `await` 后再 `setState`	默认优先级（已被恢复）	普通优先级
`startTransition(() => setState)`	Transition 优先级	低优先级，可中断
`setTimeout` / 普通 Promise 回调里的 `setState`	默认优先级	普通优先级

一句话:

`setState` 不知道“我是从 click 里出来的”，它只知道“我现在被调用时，React 全局标记的当前更新优先级是什么”。`await` 会让 React 的事件优先级标记提前结束，所以后面的 `setState` 就掉到默认优先级了。

↳ Explore React's Lane Priority System
↳ Investigate React Server Components
畅所欲问

2026/9/8 12:25:24

setState 是如何 感知 他所在 上下文环境的 更新优先级?

今天 12:17

2026/9/8 12:25:24

现在有一个问题, 好像 click点击, change监听中的setCount 更新和 startTransition中的setCount 更新具有不同的优先级.

所以问题就是 setState函数如何知道 他所在的环境?
特别是 存在异步await的情况, 比如click中 先await fetch 之后 再 setState，相比click中直接 setState的情况，前者 setState的执行环境是执行完fetch之后系统的自然调度(click早已经在await fetch 的时候就执行结束了), 而后者的setState的执行是处于click调度中的.

思考了 10s >

对，你现在问到的是 **React 优先级机制真正的核心**了。

你的理解基本正确：

> `setState` 本身并不知道"我是 click 里的 setState"。
>
> `setState` **被调用的那一刻，React 会读取当前的"更新上下文（update context）"，再根据这个上下文决定给这个 Update 分配什么 Lane。**

而 `await` 恰好会把这个上下文切断。

---

1. 先给结论

假设：

```javascript
function click() {
  setCount(count + 1);
}
```

和：

```javascript
async function click() {
  await fetch('/api');

  setCount(count + 1);
}
```

虽然这两个 `setCount` 写在同一个 `click` 函数里，但 **React 看它们的时候，所处的上下文已经完全不同。**

大致：

```
① 直接 setState

Browser click
    ↓
React event context
    ↓
click()
    ↓
setCount()
    ↓
requestUpdateLane()
    ↓
读取当前 Event Priority
    ↓
高优先级 Lane
```

而：

```
② await 后 setState

Browser click
    ↓
React event context
    ↓
click()
    ↓
await fetch()
    ↓
click 函数暂停
    ↓
React event context 结束
    ↓
click Task 结束


       ...未来 fetch 完成...


Promise continuation / microtask
    ↓
click() 从 await 后继续
    ↓
setCount()
    ↓
requestUpdateLane()
    ↓
此时已经没有原来的 click Event Priority
    ↓
Default / 其他当前上下文对应的 Lane
```

所以你说的：

> "click 已经在 await fetch 的时候执行结束了"

这个理解非常关键，而且基本就是答案。

---

2. `setState` 自己并不知道"我在哪里"

这一点一定要建立起来。

你之前看到：

```javascript
const [count, setCount] = useState(0);
```

实际上：

```
setCount
   ↓
dispatchSetState
```

它绑定的是：

```
Fiber
Queue
```

而不是：

```
click
事件类型
```

所以可以想象：

```javascript
setCount = dispatchSetState.bind(
  null,
  fiber,
  queue
);
```

调用：

```javascript
setCount(123);
```

进入：

```javascript
dispatchSetState(fiber, queue, 123)
```

然后关键的一步来了：

```javascript
const lane = requestUpdateLane(fiber);
```

也就是说：

> **Lane 是在 `setState` 被调用的瞬间，根据"当前 React 上下文"动态决定的。**

---

3. `requestUpdateLane()` 才是真正的关键

你现在应该重点盯住：

```
dispatchSetState
       ↓
requestUpdateLane
```

而不是 `setCount`。

可以把它极度简化成：

```javascript
function dispatchSetState(fiber, queue, action) {
  const lane = requestUpdateLane(fiber);

  const update = {
    lane,
    action
  };

  enqueueUpdate(queue, update);

  scheduleUpdateOnFiber(fiber, lane);
}
```

所以：

```
setCount()
   ↓
"你是什么优先级？"
   ↓
requestUpdateLane()
   ↓
Lane
```

---

4. 那 `requestUpdateLane()` 怎么知道当前是 click?

答案是：

> **React DOM 在执行事件处理器之前，临时设置了一个"当前事件更新优先级"。**

可以把 React DOM 的事件系统简化成：

```javascript
function dispatchEvent(event) {
  const previousPriority = getCurrentUpdatePriority();

  try {
    setCurrentUpdatePriority(
      getEventPriority(event.type)
    );

    // 执行你的 onClick
    dispatchEventForPluginSystem(event);

  } finally {
    setCurrentUpdatePriority(previousPriority);
  }
}
```

例如：

```
click
  ↓
DiscreteEventPriority
```

所以：

```javascript
function click() {
  setCount(count + 1);
}
```

执行到 `setCount` 的时候：

```
Current Update Priority
        ↓
DiscreteEventPriority
```

于是：

```
requestUpdateLane()
        ↓
SyncLane / 对应高优先级 Lane
```

---

5. 这其实就是一个"动态作用域"

这个概念非常重要。

React 的这种设计有点像：

```javascript
let currentPriority = null;

function runWithPriority(priority, fn) {
  const old = currentPriority;

  currentPriority = priority;

  try {
    fn();
  } finally {
    currentPriority = old;
  }
}
```

然后：

```javascript
runWithPriority('HIGH', () => {
  setState();
});
```

`setState()` 不需要知道：

```
"我是被 click 调用的"
```

它只需要问：

```javascript
currentPriority
```

得到：

```
HIGH
```

---

6. 所以 `startTransition` 本质也是干这个事情

例如：

```javascript
startTransition(() => {
  setCount(count + 1);
});
```

`startTransition` 做的事情可以粗略理解为：

```javascript
function startTransition(callback) {
  const prevTransition = currentTransition;

  currentTransition = new Transition();

  try {
    callback();
  } finally {
    currentTransition = prevTransition;
  }
}
```

于是：

```
startTransition()
       ↓
设置 currentTransition
       ↓
执行 callback
       ↓
setState()
       ↓
requestUpdateLane()
       ↓
发现 currentTransition
       ↓
TransitionLane
```

所以：

```javascript
startTransition(() => {
  setCount(...);
});
```

为什么能让 `setCount` 变成 Transition?

不是因为 `setCount` 有什么特殊能力。

而是：

```
startTransition
       ↓
改变当前 React 上下文
       ↓
setState
       ↓
读取当前上下文
       ↓
Transition Lane
```

---

7. 现在来看你提出的 `await`

这是最有意思的地方。

```javascript
async function click() {
  setCount(1);

  await fetch('/api');

  setCount(2);
}
```

实际上是两个完全不同时间点的 `setCount`。

第一个

```
click Task
│
├ React 设置:
│   currentUpdatePriority = DiscreteEventPriority
│
├ click()
│
├ setCount(1)
│    ↓
│   requestUpdateLane()
│    ↓
│   读取当前优先级
│    ↓
│   高优先级 Lane
│
└ await fetch()
        ↓
    click handler 暂停
```

然后 React DOM：

```
restore previous priority
```

最终：

```
currentUpdatePriority = NoEventPriority
```

---

8. `await fetch()` 到底发生了什么?

这里还要把 JavaScript Event Loop 加进来。

```javascript
async function click() {
  await fetch('/api');

  setCount(2);
}
```

执行：

```
click Task
│
├ fetch()
│
├ await
│
└ click 返回 Promise
        ↓
Task 结束
```

注意：

`await` 本身不会让整个 click Task 一直挂在那里。

函数被拆成：

```
click 前半部分
```

和：

```
click 后半部分
```

后半部分变成 Promise continuation。

等 `fetch()` 完成：

```
fetch Promise fulfilled
        ↓
Promise continuation
        ↓
Microtask Queue
        ↓
继续执行:
setCount(2)
```

此时：

> **原来的 click 事件上下文早就结束了。**

所以：

```
setCount(2)
```

自然不会再看到：

```
DiscreteEventPriority
```

---

9. 这就是你问题里最关键的地方

你说：

> "前者 setState 的执行环境是执行完 fetch 之后系统的自然调度(click 早已经在 await fetch 的时候就执行结束了)"

完全可以这么理解。

甚至可以画成：

```
                第一个 setState

Browser Task
┌─────────────────────────────┐
│ click                       │
│                             │
│ currentUpdatePriority = click │
│                             │
│ setState()                  │
│      ↓                      │
│   Sync/Discrete Lane        │
│                             │
│ await fetch()               │
└─────────────────────────────┘
                ↓
          click context 消失


            fetch 完成


Microtask
┌─────────────────────────────┐
│ Promise continuation        │
│                             │
│ currentUpdatePriority = none │
│                             │
│ setState()                  │
│      ↓                      │
│ Default/当前上下文 Lane      │
└─────────────────────────────┘
```

---

10. 那为什么 `startTransition` 可以改变这个?

例如：

```javascript
async function click() {
  startTransition(() => {
    setCount(1);
  });
}
```

这里：

```
startTransition
   ↓
currentTransition = transition
   ↓
setCount()
   ↓
TransitionLane
```

非常直接。

但是如果：

```javascript
async function click() {
  startTransition(async () => {
    await fetch('/api');

    setCount(1);
  });
}
```

这里就出现你问的核心问题：

```
startTransition
       ↓
currentTransition = T
       ↓
await
       ↓
callback 暂停
       ↓
startTransition 的同步作用域结束
       ↓
currentTransition 恢复
       ↓
fetch 完成
       ↓
microtask
       ↓
setCount()
```

因此 React **目前不能简单依靠普通 JavaScript 作用域自动把 Transition 上下文跨过 `await` 保存下来。**

React 官方文档目前也明确说明：

> `await` 之后的 state update 需要再次包裹 `startTransition`，这是当前的已知限制；未来 AsyncContext 可以解决这个问题。 React +1

所以：

```javascript
startTransition(async () => {
  await fetch('/api');

  // ❌ 当前不能依赖外层 transition
  setCount(1);
});
```

应该：

```javascript
startTransition(async () => {
  await fetch('/api');

  startTransition(() => {
    // ✅
    setCount(1);
  });
});
```

---

11. 这也解释了为什么 React 需要 AsyncContext

你现在已经碰到了 React 并发模型里面一个非常深的设计问题：

```
同步调用栈
```

和：

```
异步调用链
```

不是一回事。

同步的时候：

```
A
 ↓
B
 ↓
C
 ↓
setState
```

很容易：

```
currentTransition = T
```

一直沿着同步调用栈传下去。

但是：

```
A
 ↓
await
 ↓
---------------
未来某个时间
 ↓
Promise continuation
 ↓
C
 ↓
setState
```

中间已经脱离原来的 JS 调用栈了。

普通 JavaScript 变量：

```javascript
let currentTransition
```

无法天然知道：

> "这个 Promise continuation 最初是从哪个 Transition 创建出来的？"

这正是 **Async Context / AsyncLocalStorage 类机制**想解决的问题。

React 官方也把 `await` 后 Transition 丢失描述为 JavaScript 异步上下文跟踪方面的限制。 React 官方文档

---

12. 所以 React 实际上存在几种"上下文"

你现在可以把 `requestUpdateLane()` 的判断理解成：

```
                 setState
                    ↓
              requestUpdateLane()
                    ↓
          ┌─────────┴─────────┐
          ↓                   ↓
  当前有没有 Transition?   当前有没有 Event Priority?
          │                   │
          ↓                   ↓
  Transition Lane        Discrete / Continuous /
                         Default 等 Lane
```

所以：

click

```
click
 ↓
currentUpdatePriority = Discrete
 ↓
setState
 ↓
高优先级 Lane
```

`startTransition`

```
startTransition
 ↓
currentTransition = T
 ↓
setState
 ↓
Transition Lane
```

`await` 后

```
await
 ↓
原来的 event context / transition scope 消失
 ↓
Promise continuation
 ↓
setState
 ↓
重新根据"此刻"的上下文决定 Lane
```

---

13. `change` 为什么也有优先级?

你提到：

> click、change、startTransition 中的 setCount 有不同优先级。

这里需要稍微修正一下：

不是 `setCount` 自己根据函数名判断。

而是 React DOM 根据事件类型确定：

```
event type
   ↓
event priority
   ↓
currentUpdatePriority
   ↓
setState
   ↓
requestUpdateLane
```

例如概念上：

```
click
keydown
input/change
       ↓
Discrete Event Priority
       ↓
高优先级
```

而连续事件例如：

```
mousemove
scroll
pointermove
```

可能属于：

```
Continuous Event Priority
```

所以真正的链条是：

```
浏览器事件类型
        ↓
React DOM Event System
        ↓
Event Priority
        ↓
currentUpdatePriority
        ↓
setState
        ↓
requestUpdateLane
        ↓
Lane
```

---

14. 这也解释了一个非常重要的事实

同一个 `setCount` 函数，可以在不同地方产生不同 Lane。

例如：

```javascript
const [count, setCount] = useState(0);
```

同一个：

```
setCount
```

在 click：

```javascript
onClick={() => {
  setCount(1);
}}
```

可能：

```
setCount
 ↓
requestUpdateLane
 ↓
Sync / Discrete Lane
```

在 transition：

```javascript
startTransition(() => {
  setCount(2);
});
```

则：

```
setCount
 ↓
requestUpdateLane
 ↓
Transition Lane
```

在 timeout：

```javascript
setTimeout(() => {
  setCount(3);
}, 1000);
```

则：

```
setCount
 ↓
requestUpdateLane
 ↓
Default Lane
```

在 await 后：

```javascript
async function foo() {
  await fetch(...);

  setCount(4);
}
```

通常也是：

```
setCount
 ↓
requestUpdateLane
 ↓
Default Lane
```

所以：

> **Lane 是 Update 的属性，而不是 `setCount` 函数的固定属性。**

这一点非常重要。

---

15. 你现在可以重新理解你截图里的调用栈

你之前截图看到：

```
increment
  ↓
dispatchSetState
  ↓
dispatchSetStateInternal
  ↓
scheduleUpdateOnFiber
  ↓
ensureRootIsScheduled
```

其实在：

```
dispatchSetState
```

里面最值得你下一步单步进去的是：

```javascript
const lane = requestUpdateLane(fiber);
```

因为真正的核心问题：

> **"这次 setState 到底是什么优先级？"**

就是在这里回答的。

可以把整个 React 19 更新过程压缩成：

```
                    setState()
                        │
                        ↓
                 dispatchSetState
                        │
                        ↓
                requestUpdateLane
                        │
            ┌───────────┴───────────┐
            │                       │
    currentTransition?      current event priority?
            │                       │
            ↓                       ↓
    Transition Lane             Event Lane
            │                       │
            └───────────┬───────────┘
                        ↓
                     Update
                        │
                        ↓
               scheduleUpdateOnFiber
                        │
                        ↓
               ensureRootIsScheduled
                        │
                        ↓
                    Scheduler
                        │
                        ↓
                     Render
                        │
                        ↓
                     Commit
```

而 `await` 的本质，就是让 `setState()` 的第二次调用发生在另一个 JavaScript continuation 中，因此它不再天然继承原来的 click Event Priority。

这也是为什么你现在研究 `requestUpdateLane()` 是非常正确的方向。你会发现：**React 的"优先级"本质不是 `setState` 记住的，而是 React 在每一次 `setState` 调用现场，根据当时的上下文重新决定的。**

ChatGPT 也可能会犯错。请核查重要信息。